Department of Computer Science

German Traffic Sign Recognition Benchmark Report

Alvin Momoh
Jash Thakkar
Mamadou Diallo
Doron Yospa

April 30, 2026

Abstract

This project focuses on traffic sign recognition using machine learning and deep learning models. The goal is to classify traffic sign images correctly using the German Traffic Sign Recognition Benchmark dataset. Traffic sign recognition is important because it supports self-driving cars, driver assistance systems, and road safety technology.

Several models were implemented and compared in this project, including K-Nearest Neighbors (KNN), Random Forest, HOG with Linear SVC, and ResNet18. The dataset was processed by resizing images, normalizing pixel values, and splitting the data into training, validation, and testing sets. ResNet18 used transfer learning, where a pretrained neural network was adapted to classify 43 traffic sign classes.

The models were evaluated using accuracy, precision, recall, F1-score, and confusion matrix. The results showed that Random Forest achieved the highest validation accuracy, while ResNet18 also performed strongly and showed good generalization. Overall, the project shows that both optimized traditional machine learning models and deep learning models can perform well on traffic sign recognition, but ResNet18 is more scalable for real-world image classification tasks.

Keywords: Traffic Sign Recognition, ResNet18, Machine Learning, Deep Learning, Computer Vision

Report's total word count: 3891 words

Contents

List of Figures iv

List of Tables v

1 Introduction 1
- 1.1 Background . . . 1
- 1.2 Problem Statement . . . 1
- 1.3 Aims and Objectives . . . 2
- 1.4 Solution Approach . . . 2
- 1.5 Summary of Contributions and Achievements . . . 2
- 1.6 Organization of the Report . . . 3

2 Literature Review 4
- 2.1 Traffic Sign Recognition . . . 4
- 2.2 K-Nearest Neighbors . . . 4
- 2.3 Random Forest . . . 4
- 2.4 HOG with Linear SVC . . . 5
- 2.5 Convolutional Neural Networks and ResNet18 . . . 5
- 2.6 Summary . . . 5

3 Methodology 7
- 3.1 Dataset . . . 7
- 3.2 Data Preprocessing . . . 7
- 3.3 Models Used . . . 7
  - 3.3.1 K-Nearest Neighbors (KNN) . . . 7
  - 3.3.2 Random Forest . . . 7
  - 3.3.3 HOG with Linear SVC . . . 8
  - 3.3.4 ResNet18 . . . 8
- 3.4 ResNet18 Training Process . . . 8
  - 3.4.1 Stage 1: Training the Final Layer . . . 8
  - 3.4.2 Stage 2: Fine-Tuning . . . 8
- 3.5 Evaluation Metrics . . . 8
- 3.6 Implementation Tools . . . 8
- 3.7 Summary . . . 9

4 Results 10
- 4.1 Overview of Results . . . 10
- 4.2 Model Performance Comparison . . . 10
- 4.3 ResNet18 Final Evaluation . . . 10

4.4 Confusion Matrix 11
4.5 Classification Report 12
4.6 Summary of Results 13

5 Discussion and Analysis **14**
5.1 Comparison of Model Performance 14
5.2 Significance of the Findings 14
5.3 Limitations 15
5.4 Interpretation of Confusion Matrix 15
5.5 Summary 15

6 Conclusions and Future Work **16**
6.1 Conclusion 16
6.2 Future Work 16

References **18**

List of Figures

4.1 Confusion Matrix of ResNet18 11

List of Tables

4.1 Model Performance Comparison 10
4.2 Final ResNet18 Evaluation Metrics 11

List of Abbreviations

PCA	Principal Component Analysis

Chapter 1

Introduction

1.1 Background

Traffic sign recognition is an important task in computer vision. The purpose of traffic sign recognition is to identify and classify traffic signs from images or video. This technology is useful in self-driving cars, driver assistance systems, and intelligent transportation systems. A vehicle that can recognize signs correctly can understand speed limits, warnings, stop signs, and other road instructions.

Automation is becoming more common in daily life, and transportation is one of the main areas where this is happening. Companies that work on autonomous driving need computer vision systems that can recognize the road environment. These systems must understand lanes, vehicles, pedestrians, traffic lights, and traffic signs. Traffic signs are especially important because they communicate rules that drivers must follow. If an automated system fails to recognize a traffic sign, it may make an unsafe decision.

Traffic sign recognition is not always easy. Images can be blurry, dark, tilted, small, or partly blocked. Some signs may also look similar to each other. For example, different speed limit signs may have the same shape but different numbers. Warning signs may also have similar colors and shapes. Because of this, the model must learn details such as color, edges, shape, and inner symbols.

This project uses the German Traffic Sign Recognition Benchmark dataset. The dataset contains many traffic sign images with class labels. Each image belongs to one of the traffic sign classes. The task is to train models that can correctly predict the class of each image. This makes the project a supervised image classification problem.

1.2 Problem Statement

The main problem in this project is to classify traffic sign images accurately. The model receives an image as input and must predict the correct traffic sign class. This problem is challenging because the images are not always perfect. Some images may have different lighting conditions, different angles, blur, or noise. These problems can make it harder for a model to understand the sign.

Traditional machine learning models may struggle because they often depend on hand-crafted features or flattened pixel values. For example, KNN compares images directly, so it can be affected by small changes in brightness or position. HOG with Linear SVC looks at edge directions, so it may be affected when an image is blurry or rotated. Random Forest can handle noise better, but it still does not learn visual features in the same way as a convolutional

neural network.

Deep learning models such as ResNet18 can learn features automatically from images. This makes them useful for image classification problems. However, deep learning models may require more training time and computational power. Therefore, this project compares both traditional machine learning models and a deep learning model to understand which approach works best for this dataset.

1.3 Aims and Objectives

The main aim of this project is to build and compare different models for traffic sign recognition. The goal is not only to get high accuracy, but also to understand why some models perform better than others.

The objectives of this project are:

- To preprocess the German Traffic Sign Recognition Benchmark dataset.
- To train and evaluate different models, including KNN, Random Forest, HOG with Linear SVC, and ResNet18.
- To compare the models using accuracy, precision, recall, F1-score, and confusion matrix.
- To identify which models generalize well to unseen traffic sign images.
- To explain the strengths and weaknesses of each model.

1.4 Solution Approach

The project followed a structured machine learning workflow. First, the dataset was loaded and inspected. The dataset contains image paths and class labels. The images were resized and normalized so that the models could process them in a consistent way. The data was then split into training, validation, and testing sets.

Several models were trained. KNN was used as a baseline model. Random Forest was used because it is better at handling noisy data than KNN. HOG with Linear SVC was used because HOG extracts edge and gradient information from images. ResNet18 was used as the deep learning model because it is a convolutional neural network designed for image classification.

The ResNet18 model used transfer learning. This means that the model started with pretrained weights from a large image dataset. The final classification layer was replaced so that the model could predict the traffic sign classes. The model was trained in two stages. In the first stage, only the final classification layer was trained. In the second stage, deeper layers were fine-tuned using a smaller learning rate.

1.5 Summary of Contributions and Achievements

This project implemented and compared several approaches for traffic sign recognition. The project showed how traditional machine learning methods and deep learning methods behave on the same dataset. It also demonstrated the use of transfer learning with ResNet18.

The main achievement was building a working ResNet18 model that can classify traffic sign images and output evaluation metrics such as accuracy, precision, recall, F1-score, classification report, and confusion matrix. The results also showed that Random Forest achieved very strong validation performance, while ResNet18 performed well and provided a scalable deep learning approach.

1.6 Organization of the Report

This report is organized into six chapters. Chapter 1 introduces the project, explains the problem, and describes the aims and objectives. Chapter 2 reviews the main models used in the project. Chapter 3 explains the dataset, preprocessing steps, model implementation, and evaluation metrics. Chapter 4 presents the results. Chapter 5 discusses and analyzes the results. Chapter 6 concludes the report and suggests future work.

Chapter 2

Literature Review

2.1 Traffic Sign Recognition

Traffic sign recognition is a computer vision task where a model identifies traffic signs from images. It is important in autonomous driving and driver assistance systems because traffic signs provide road instructions. A reliable traffic sign recognition system can help a vehicle understand speed limits, warnings, and road rules.

Traffic sign recognition is usually treated as an image classification problem. In image classification, the model receives an image and predicts one label. In this project, each label represents a traffic sign class. The German Traffic Sign Recognition Benchmark (GTSRB) dataset contains multiple traffic sign classes, and each image has a correct class label.

There are different ways to solve this problem. Traditional machine learning methods use manually selected features or flattened pixel values. Deep learning methods learn features directly from images. Both approaches have advantages and disadvantages.

2.2 K-Nearest Neighbors

K-Nearest Neighbors, also called KNN, is a simple classification model. It predicts the class of a new example by comparing it to examples in the training set. The model finds the nearest examples and chooses the class that appears most often among them.

KNN is easy to understand and can be used as a baseline model. In this project, KNN was useful because it gave a starting point for comparison. However, KNN can struggle with image data because images are high-dimensional. If the model uses flattened pixel values, small changes in brightness, blur, or position can change the distance between images. This can reduce performance.

Another issue with KNN is computation time. During prediction, KNN often compares the test image to many training images. This can become slow when the dataset is large. For traffic sign recognition, KNN can work reasonably well, but it is not always the best choice for real-time systems.

2.3 Random Forest

Random Forest is a machine learning model that uses many decision trees. Each tree makes a prediction, and the final result is based on the combined output of all trees. This makes Random Forest more stable than a single decision tree.

Random Forest is useful because it can handle noisy data better than KNN. Instead of comparing each image directly to other images, Random Forest learns decision patterns from the data. In this project, Random Forest performed very well, especially after tuning. The model was able to achieve high validation accuracy.

However, Random Forest still has limitations for image classification. It usually works with flattened image features or extracted features. This means it does not naturally understand spatial relationships in the same way as a convolutional neural network. For example, a CNN can learn that nearby pixels form edges and shapes. Random Forest may not capture this type of spatial structure as well.

2.4 HOG with Linear SVC

Histogram of Oriented Gradients, or HOG, is a feature extraction method used in computer vision. HOG describes an image by looking at the direction of edges and gradients. Instead of using raw pixels directly, it converts the image into a feature vector based on shape and edge information.

Linear SVC is a support vector classifier that separates classes using linear decision boundaries. When HOG is combined with Linear SVC, the model uses edge-based image features for classification. This can be useful for traffic signs because traffic signs often have clear shapes, symbols, and borders.

In this project, HOG with Linear SVC performed reasonably well but did not outperform the best models. One reason is that traffic sign images may contain noise, blur, and angle distortion. These problems can make edge and gradient information less reliable. Also, if images are resized to a small resolution, HOG may lose some useful detail.

2.5 Convolutional Neural Networks and ResNet18

Convolutional Neural Networks, or CNNs, are deep learning models designed for image data. CNNs use convolutional layers to learn visual patterns such as edges, corners, textures, shapes, and objects. This makes CNNs very effective for image classification.

ResNet18 is a CNN architecture that uses residual connections. These connections allow information to pass through the network more easily. This helps the model train deeper networks and reduces the problem of vanishing gradients. ResNet18 is smaller than deeper ResNet models, but it is still powerful enough for many image classification tasks.

In this project, ResNet18 was used with transfer learning. Transfer learning means using a model that was already trained on a large dataset and adapting it to a new task. This helps because the early layers of the model have already learned useful visual features. The final layer can then be changed to match the number of classes in the traffic sign dataset.

ResNet18 is useful for this project because it can learn directly from image patterns. It does not require manual feature extraction like HOG. It can also learn more complex relationships than KNN or Random Forest. The main disadvantage is that it usually needs more training time, especially on a CPU.

2.6 Summary

The models used in this project represent different approaches to image classification. KNN is simple and useful as a baseline. Random Forest is stronger and handles noise well. HOG with

Linear SVC uses edge-based features but can struggle with noisy images. ResNet18 learns features directly from images and provides strong performance.

Overall, deep learning models such as ResNet18 are more suitable for complex image classification tasks because they can learn both low-level and high-level features automatically. However, traditional models such as Random Forest can still achieve strong results depending on the dataset.

Chapter 3

Methodology

3.1 Dataset

The dataset used in this project is the German Traffic Sign Recognition Benchmark (GTSRB) Stallkamp et al. (2011), obtained from Kaggle Kaggle (2023)..This dataset contains images of traffic signs along with their corresponding class labels. Each image is associated with a ClassId, which represents the correct traffic sign category.

The dataset includes multiple traffic sign classes, making it suitable for a multi-class classification problem. The images vary in size, lighting, orientation, and quality, which makes the classification task more challenging.

3.2 Data Preprocessing

Before training the models, the dataset was preprocessed to ensure consistent input.

The following steps were applied:

- Images were resized to 224×224 pixels to match the input size required by ResNet18.
- Pixel values were normalized using ImageNet mean and standard deviation.
- The dataset was split into training, validation, and testing sets.

The training set was used to train the models, the validation set was used to tune the models, and the test set was used for final evaluation.

3.3 Models Used

Several models were implemented and compared in this project.

3.3.1 K-Nearest Neighbors (KNN)

KNN was used as a baseline model. It classifies images by comparing them to the nearest training examples. Different values of K were tested to improve performance.

3.3.2 Random Forest

Random Forest was used because it is robust to noise and can handle complex patterns. The model uses multiple decision trees and combines their predictions.

3.3.3 HOG with Linear SVC

HOG was used to extract edge and gradient features from images. These features were then used with a Linear Support Vector Classifier (SVC) for classification.

3.3.4 ResNet18

ResNet18 is a convolutional neural network used for image classification. It was selected because it can learn image features automatically and perform well on complex image datasets.

3.4 ResNet18 Training Process

The ResNet18 model was trained using transfer learning. A pretrained ResNet18 model was loaded, and its final classification layer was replaced to match the number of traffic sign classes.

The training was performed in two stages:

3.4.1 Stage 1: Training the Final Layer

In this stage, all pretrained layers were frozen. Only the final classification layer was trained. This allowed the model to learn the traffic sign classes without modifying the pretrained feature layers.

3.4.2 Stage 2: Fine-Tuning

In this stage, the deeper layers of the network were unfrozen and fine-tuned using a smaller learning rate. This allowed the model to adapt better to the dataset.

3.5 Evaluation Metrics

The models were evaluated using several metrics:

- **Accuracy**: Measures the percentage of correct predictions.
- **Precision**: Measures how many predicted positives are correct.
- **Recall**: Measures how many actual positives are correctly identified.
- **F1-score**: A balance between precision and recall.
- **Confusion Matrix**: Shows correct and incorrect predictions for each class.

3.6 Implementation Tools

The project was implemented using Python. PyTorch and Torchvision were used to build and train the ResNet18 model. Scikit-learn was used for evaluation metrics such as accuracy, precision, recall, F1-score, and confusion matrix. Matplotlib was used to generate graphs and visualize results.

3.7 Summary

This chapter described the dataset, preprocessing steps, models used, and training process. The methodology provides the foundation for evaluating the performance of different models, which is presented in the next chapter.

Chapter 4

Results

4.1 Overview of Results

This chapter presents the performance of the models used for traffic sign recognition. The models were evaluated using accuracy, precision, recall, and F1-score, along with the confusion matrix. The goal is to compare traditional machine learning models with a deep learning model.

The models tested in this project include K-Nearest Neighbors (KNN), Random Forest, HOG with Linear SVC, and ResNet18.

4.2 Model Performance Comparison

Table 4.1 shows the performance of the models based on validation accuracy.

Table 4.1: Model Performance Comparison

Model	Configuration	Validation Accuracy
KNN	K = 5 (Gray)	0.8498
KNN	K = 3 (Gray)	0.8731
KNN	K = 3 (RGB)	0.8991
Random Forest	Gray	0.9742
Random Forest	RGB	0.9759
HOG + Linear SVC	Tuned	0.8876
ResNet18	Initial	0.9246
ResNet18	Tuned	0.9483

The results show that Random Forest achieved the highest validation accuracy of approximately 97.5%. The tuned ResNet18 model also performed strongly during experimentation. KNN achieved moderate performance, while HOG with Linear SVC performed slightly lower.

4.3 ResNet18 Final Evaluation

The ResNet18 model was further evaluated using additional metrics obtained from the evaluation script.

The final ResNet18 evaluation achieved an accuracy of 0.9053, with precision of 0.9115, recall of 0.9053, and F1-score of 0.9053. These results show that the model performs well overall, with high precision indicating reliable predictions and balanced recall showing good coverage of all classes.

Table 4.2: Final ResNet18 Evaluation Metrics

Metric	Value
Accuracy	0.9053
Precision	0.9115
Recall	0.9053
F1-score	0.9053
Prediction Time	519.45 s

The accuracy measures the overall correctness of the model. Precision shows how reliable the predictions are. Recall measures how well the model identifies all relevant instances. The F1-score provides a balance between precision and recall. Prediction time reflects the efficiency of the model during inference.

4.4 Confusion Matrix

The confusion matrix provides a detailed view of the model's performance across all classes.

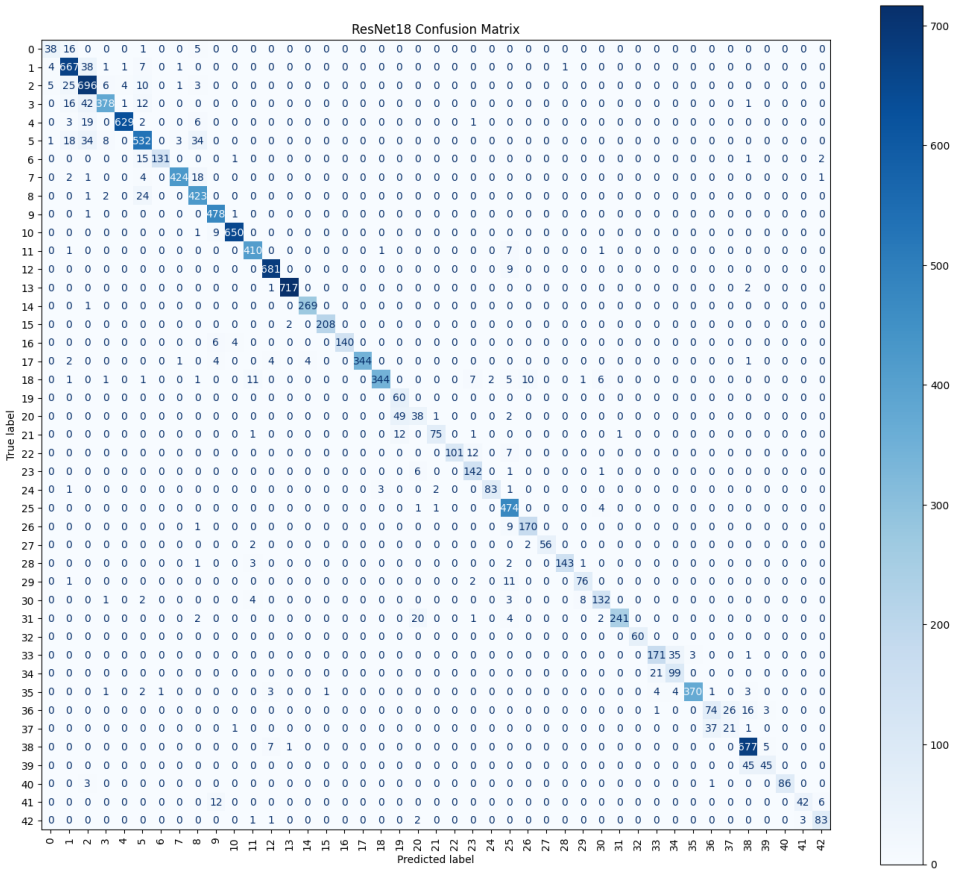

Figure 4.1: Confusion Matrix of ResNet18

Figure 4.1 shows that most predictions are correct, with values concentrated along the diagonal. Some misclassifications occur between visually similar traffic signs, which indicates that these classes are more difficult to distinguish.

4.5 Classification Report

The classification report provides precision, recall, and F1-score for each class.

```
 ========== EVALUATION RESULTS ==========
Accuracy        : 0.9053
Precision       : 0.9115
Recall          : 0.9053
F1-Score        : 0.9053
Prediction Time : 519.45 seconds

Classification Report:
              precision    recall  f1-score   support

           0     0.5000    0.6500    0.5652        60
           1     0.8562    0.9097    0.8822       720
           2     0.8784    0.8667    0.8725       750
           3     0.9614    0.7756    0.8585       450
           4     0.9682    0.9227    0.9449       660
           5     0.8326    0.8762    0.8538       630
           6     0.9927    0.9067    0.9477       150
           7     0.9599    0.9578    0.9588       450
           8     0.8554    0.9467    0.8987       450
           9     0.9477    0.9812    0.9642       480
          10     0.9865    0.9939    0.9902       660
          11     0.9587    0.9405    0.9495       420
          12     1.0000    0.9333    0.9655       690
          13     0.9676    0.9944    0.9808       720
          14     1.0000    1.0000    1.0000       270
          15     1.0000    0.9810    0.9904       210
          16     0.9932    0.9667    0.9797       150
          17     1.0000    0.9528    0.9758       360
          18     0.9886    0.8923    0.9380       390
          19     0.5714    0.3333    0.4211        60
          20     0.5899    0.9111    0.7162        90
          21     0.9718    0.7667    0.8571        90
          22     1.0000    0.8083    0.8940       120
          23     0.8304    0.9467    0.8847       150
          24     0.9277    0.8556    0.8902        90
          25     0.8825    0.9542    0.9169       480
          26     0.8571    0.9667    0.9086       180
          27     0.7692    1.0000    0.8696        60
          28     0.8516    0.8800    0.8656       150
          29     0.9265    0.7000    0.7975        90
          30     0.8983    0.7067    0.7910       150
          31     0.9373    0.9963    0.9659       270
          32     0.9677    1.0000    0.9836        60
          33     0.9773    0.6143    0.7544       210
          34     0.6257    0.9750    0.7622       120
          35     0.9611    0.9513    0.9562       390
```

36	0.5490	0.4667	0.5045	120
37	0.3051	0.3000	0.3025	60
38	0.8936	0.9246	0.9088	690
39	0.4587	0.5556	0.5025	90
40	0.9032	0.9333	0.9180	90
41	0.6506	0.9000	0.7552	60
42	0.8171	0.7444	0.7791	90
accuracy			0.9053	12630
macro avg	0.8551	0.8521	0.8470	12630
weighted avg	0.9115	0.9053	0.9053	12630

The classification report shows detailed performance for each class. Most classes achieved high precision and recall, indicating strong performance. However, some classes such as class 0, 19, and 37 show lower scores, which suggests that these classes are more difficult to classify due to similarity or limited data.

The curves show that both training and validation accuracy improved over time, and the model achieved stable performance after fine-tuning. There is no strong evidence of overfitting, as validation accuracy follows the training accuracy closely.

4.6 Summary of Results

The results show that Random Forest achieved the highest validation accuracy, while ResNet18 demonstrated strong performance and better scalability for image classification tasks. KNN provided a useful baseline, and HOG with Linear SVC showed moderate performance.

Overall, deep learning models such as ResNet18 are more suitable for complex image classification tasks, especially when dealing with noisy and high-dimensional data.

Chapter 5

Discussion and Analysis

5.1 Comparison of Model Performance

The results show that different models performed differently on the traffic sign recognition task. KNN gave a useful baseline, but it was not the strongest model. Random Forest performed very well and achieved the highest validation score. HOG with Linear SVC improved after tuning, but it did not perform as strongly as Random Forest or ResNet18. ResNet18 performed strongly and showed that deep learning is effective for traffic sign image classification.

KNN was limited because it compares images directly. When images are noisy, blurry, or slightly shifted, the pixel values can change a lot. This makes it harder for KNN to compare images accurately. Lowering K improved performance at first, but lowering it too much can make the model overfit.

Random Forest performed strongly because it combines many decision trees. This helps reduce the effect of noise. It also learns decision patterns instead of relying only on direct image distance. However, Random Forest can still overfit when the training accuracy becomes perfect and the validation accuracy does not improve as much.

HOG with Linear SVC was expected to perform well because traffic signs often have strong shapes and edges. However, the model struggled more than expected. One possible reason is that many images contain blur, angle distortion, or lighting problems. These problems can make edge information less clear. Also, if images are resized to smaller dimensions, HOG may lose some details.

ResNet18 performed well because it learns features directly from images. It can learn low-level features such as edges and colors, and it can also learn higher-level features such as shapes and symbols. Because ResNet18 uses convolutional layers, it is better suited for image tasks than models that use flattened image data.

5.2 Significance of the Findings

The findings are important because they show that traffic sign recognition can be solved using several approaches. Traditional models can perform well when the dataset is clean and the features are useful. Random Forest showed that classical machine learning can still be very powerful.

However, ResNet18 is more scalable for real-world image recognition. In real driving environments, traffic signs may appear in many different conditions. A deep learning model can continue learning from more data and can be improved with more training, better augmentation, and stronger architectures.

The strong performance of ResNet18 also shows the value of transfer learning. Instead of training a CNN from zero, the model used pretrained weights. This made training more effective because the model already had useful visual knowledge. Fine-tuning helped the model adapt to the traffic sign dataset.

5.3 Limitations

There were several limitations in this project. First, training on a CPU was slow. Deep learning models usually train much faster on a GPU. Because of limited computation power, training and tuning ResNet18 took more time.

Second, the dataset contains images with noise, blur, and different lighting conditions. These issues can reduce model performance. Some signs are also visually similar, which can cause confusion between classes.

Third, not all possible CNN architectures were tested. ResNet18 performed well, but deeper models such as ResNet50 or EfficientNet may perform better. More experiments would be needed to confirm this.

Fourth, some traditional models may have benefited from more tuning. For example, HOG with Linear SVC could be improved by testing more HOG parameters, image sizes, and regularization values.

5.4 Interpretation of Confusion Matrix

The confusion matrix helps explain where the model made mistakes. Correct predictions appear on the diagonal. If most values are on the diagonal, the model is performing well. Values outside the diagonal show misclassifications.

For traffic sign recognition, mistakes often happen between signs that have similar colors, shapes, or symbols. For example, signs with similar circular shapes may be confused. Images with blur or poor lighting can also lead to wrong predictions.

The confusion matrix is useful because accuracy alone does not show which classes are difficult. A model may have high accuracy overall but still perform poorly on some smaller classes. The classification report and confusion matrix together provide a clearer view of model performance.

5.5 Summary

Overall, the project showed that Random Forest and ResNet18 were the strongest models. Random Forest achieved the highest validation score, while ResNet18 showed strong performance and better scalability. The results suggest that ResNet18 is a strong choice for future work because it learns visual features directly from the images and can improve with more data and better hardware.

Chapter 6

Conclusions and Future Work

6.1 Conclusion

This project investigated traffic sign recognition using several machine learning and deep learning models. The main goal was to classify traffic sign images correctly using the German Traffic Sign Recognition Benchmark dataset. The project tested KNN, Random Forest, HOG with Linear SVC, and ResNet18.

The project showed that traffic sign recognition is a challenging task because traffic sign images can vary in lighting, angle, blur, and quality. Some signs also look similar, which can cause misclassification. Because of this, it is important to use models that can handle noise and learn useful features.

KNN worked as a baseline model. It performed reasonably well, but it was sensitive to noise and image differences. Random Forest performed very strongly and achieved the highest validation score. This showed that Random Forest can handle noisy image data well when the features are useful.

HOG with Linear SVC used edge and gradient features. It improved after tuning, but it did not perform as strongly as Random Forest or ResNet18. This suggests that edge-based features alone may not be enough for this dataset, especially when images are blurry or distorted.

ResNet18 performed strongly and showed the value of deep learning for image classification. The model used transfer learning, which allowed it to start with pretrained image features. After replacing the final classification layer and fine-tuning deeper layers, the model was able to classify traffic signs effectively.

Overall, Random Forest achieved the best validation performance, while ResNet18 was the most scalable deep learning approach. ResNet18 is especially useful for real-world image recognition because it learns visual patterns directly from images and can be improved with more data, better augmentation, and stronger hardware.

6.2 Future Work

There are several ways this project could be improved in the future. First, more CNN architectures could be tested. ResNet18 performed well, but models such as ResNet50, EfficientNet, MobileNet, and DenseNet may produce better results. These models could be compared to determine which deep learning architecture works best for traffic sign recognition.

Second, training should be done on a GPU. Training on CPU is slow, especially for deep learning models. A GPU would make it possible to train for more epochs, test more hyperpa-

rameters, and run more experiments in less time.

Third, more data augmentation could be used. Traffic signs in real life may appear with different lighting, angles, zoom levels, and weather conditions. Augmentation techniques such as rotation, color jitter, random affine transformations, cropping, and brightness changes can help the model generalize better.

Fourth, the model could be tested on real-world images outside the dataset. This would show whether the model works well beyond the controlled benchmark dataset. Real driving images may contain more background noise, motion blur, and occlusion.

Finally, the trained model could be deployed into a small application. For example, a user could upload a traffic sign image, and the system could predict the traffic sign class. This would make the project more practical and demonstrate how the model could be used in real-world systems.

References

Kaggle (2023), 'Gtsrb - german traffic sign recognition benchmark dataset', https://www.kaggle.com/datasets/meowmeowmeowmeowmeow/gtsrb-german-traffic-sign. Accessed: 2026.

Stallkamp, J., Schlipsing, M., Salmen, J. and Igel, C. (2011), The german traffic sign recognition benchmark: A multi-class classification competition, *in* 'IEEE International Joint Conference on Neural Networks'.